

Comprehensive Parametric Sensitivity, Statistical Validation, and Security Performance Tradeoff Modeling of Modern Password Hashing Algorithms

Divyashree Duggegowda¹, A. Syed Haroon², Sai Thejas G S³, Shashank D³,
Bharath Prakash³, and L Shri Ragavendra Prasad³

¹ Assistant Professor, Presidency School of Information Science, Presidency
University, Bengaluru, Karnataka, India
divyankushi@gmail.com

² Head & Senior Lecturer, Department of Computer Science, School of Science and
Information Technology (SSIT), Skyline University Nigeria
syedharoonmca@gmail.com

³ Students, Presidency School of Information Science, Presidency University,
Bengaluru, Karnataka, India
SAI.20242MCA0220@presidencyuniversity.in,
RAGAVENDRA.20242MCA0222@presidencyuniversity.in,
BHARATH.20242MCA0230@presidencyuniversity.in,
SHASHANK.20242MCA0182@presidencyuniversity.in

Abstract. Increased reliance on computer technology led to the emergence of secure password storage as an important issue, with the difficulty of finding a balance between computing efficiency and the ability to resist modern-day attacks. The task remains hard because previously known methods were unable to provide both. Several algorithms exist today including SHA-256, bcrypt, PBKDF2, and Argon2, yet none of them combines all desired properties. SHA-256 is one of the most computationally efficient methods yet lacks any attack resistant property. The next two methods increase attack resistance but at the expense of adding additional computational effort. Furthermore, neither algorithm possesses memory hardness, which is essential in order to withstand hardware attacks. All that makes it necessary to develop a comprehensive index of evaluation of different password hashing algorithms. Currently, methods for analysis of various passwords are limited to specific factors, such as speed or cost. As opposed to that, the proposed Security Efficiency Index (SEI) integrates all aspects, including resistance to dictionary, brute force attacks, memory hardness, and computational delay. Experiments show that even though SHA-256 is highly efficient yet highly vulnerable to brute force attacks, Argon2 outperforms other algorithms when tested according to various SEI factors.

Keywords: Argon2 · bcrypt · Brute-force resistance · Cryptographic security · Memory hardness · Password hashing algorithms · PBKDF2 · Security Efficiency Index · Security-performance tradeoff · SHA-256

1 Introduction

Password authentication is the most commonly employed mode of verifying users on the Internet, within various software applications, within organizational intranets, within banking applications, and within cloud computing. EPassword authentication is the most commonly employed mode of verifying users on the Internet, within various software applications, within organizational intranets, within banking applications, and within cloud computing. Even with improvements like multi-factor authentication and security tokens, the usage of passwords has not decreased and passwords continue to be the main and sometimes the only means of authentication [7]. In recent times, massive credential breaches in the last decade have consistently demonstrated the devastating impact of inadequate password security measures. Breaches involving hundreds of millions of unsalted or weakly hashed credentials from prominent organizations have demonstrated the gap between password security theory and practice [5]. In the case of exfiltrated password databases, the security of the credentials depends on the computational and memory costs imposed by the hashing algorithm. Modern attackers exploit GPU acceleration and parallel architectures, making memory-hard hashing essential [4]. GPU-based optimizations significantly accelerate password cracking processes [11]. Advanced parallel cracking techniques further amplify attack efficiency [12]. If algorithms are not capable of imposing a high enough adversary cost, it can result in rapid cracking even after the detection and remediation of the initial breach. Recent large-scale credential breaches have demonstrated weaknesses in password security practices [7]. Probabilistic password cracking models further expose vulnerabilities in real-world password datasets [13]. Deep learning-based password guessing techniques further enhance adversarial capabilities in practical attack scenarios [16].

1.1 Research Gap

Nevertheless, in the presence of a number of adaptive KDFs, there exist a number of gaps in the password hashing literature. In most of the comparative password hashing literature, benchmarking has been performed on a single parameter, i.e., execution time/throughput, without considering memory consumption, brute force scalability, and use of system resources in an integrated manner. This is because most of the password hashing literature has followed a reductionist approach, which is not enough in understanding the multidisciplinary security of password hashing algorithms [1]. In addition, multidisciplinary parametric sensitivity analysis of password hashing algorithms, in which memory cost, iteration count, and parallelism are varied in an integrated manner, has not been addressed properly in the password hashing literature. In most guidelines, only minimum values of parameters are provided. However, there exists a lack of quantitative characterization of parameter space. Recent studies highlight the absence of unified frameworks integrating both performance and security metrics [3]. Security-performance trade-offs in cryptographic systems require more comprehensive evaluation models [10].

Real-world deployment studies indicate that practitioners often lack clear guidance on parameter tuning for password hashing algorithms, leading to inconsistent configurations [14].

2 Literature Review

Alex Biryukov et al. (2021) this research paper proposes Argon2 as a consistent memory-hard password hashing algorithm, showing prominence on emphasizing tunable parameters for managing security and performances, stressing its resistance to GPU-based attacks. Wenbo Bai and Jeremiah Blocki (2021) proposed DAHash, optimizes parameter tuning based on password distribution by improving cost proficiency while guaranteeing strong security. Jeremiah Blocki et al. (2019) proposed stalwart constructions to enhance robustness against adversarial optimization techniques by discovering new attack vectors and worked on data-independent memory hard functions. Oluwaseun Etese and Adebayo Adesina (2025) worked on better scalability through a comparative analysis of Argon2 and Bcrypt, revealing Argon2’s exceptional resistance to advanced decrypting techniques. The National Institute of Standards and Technology (2024) functioned on advanced authentication security by providing upgraded digital identity principles, highlighting memory-hard hashing algorithms and secure password storage.

3 Proposed Evaluation Framework

A systematic analysis of password hashing functions needs an evaluation scheme that considers security and efficiency aspects together. Existing methods based on individual parameters like computation time or bandwidth fall short when it comes to dealing with contemporary attack strategies. Offline attacks on compromised credentials are enabled by parallel processing and memory architectures, making it clear that a new approach is needed. In order to assess the efficiency of a password hash function, a framework for evaluation must be devised to measure both security and efficiency aspects of the algorithms. Traditional metrics such as runtime and throughput fail to serve this purpose, as contemporary adversary models utilize modern capabilities of parallelization and memory to launch attacks against passwords. Such approaches fail to accurately reflect adversarial capabilities in modern environments [15] Such approaches fail to accurately reflect adversarial capabilities in modern environments [2]. In order to meet this challenge, a framework is developed to evaluate algorithms based not only on their empirical runtime but also on a simulated adversary cost model. The proposed framework evaluates algorithms under various settings, such as different numbers of iterations, memory size, and level of parallelization. In particular, the measurement of runtime characteristics is performed through analysis of an algorithm’s latency, memory usage, and scalability.

3.1 Security Efficiency Index (SEI)

The Security Efficiency Index (SEI) forms the heart of the above framework and is essentially an index that encapsulates the relationship between security and efficiency. SEI takes into account three important parameters, namely:

$$SEI = \frac{B \times M}{T} \quad (1)$$

where:

- $B \in \mathbb{R}^+$ stands for the brute force multiplier, indicating the extent to which computational work increases when faced with an attacker,
- $M \in \mathbb{R}^+$ stands for memory use in MB, indicating how much memory is used during each instance of hashing,
- $T \in \mathbb{R}^+$ indicates average execution time in seconds, which corresponds to computational latency.

This equation embodies the idea of increasing the cost of an attack while retaining functionality. Multiplication of B and M accounts for the overall cost to an attacker in terms of computing and memory, and division by T ensures that highly inefficient setups are not rewarded. This formulation aligns with modern cryptographic practices that combine computational and memory costs for secure password hashing [4]. This formulation is also supported by adversarial cost modeling approaches [2].

3.2 Rationale and Adversarial Motivation

The development of SEI is based on cost modeling, which involves maximizing hash computations by adversaries considering hardware limits. Exhaustive search costs for passwords within space $|\mathcal{P}|$ can be calculated as:

$$C_{\text{attack}} = \frac{|\mathcal{P}| \cdot T}{P_{\text{eff}}} \quad (2)$$

Password cracking efficiency is significantly influenced by probabilistic modeling techniques [2].

Where P_{eff} is the effective parallelism?

The memory-hardness aspect is achieved by making the P_{eff} decrease by increasing memory demands per thread, hence reducing the number of threads that can compute at once using parallel hardware like GPUs. This explains why it is necessary to consider memory (M) as a multiplicative component in SEI.

3.3 Formal Sensitivity Analysis

For examining the effects of each variable, the first partial derivatives of SEI are as follows:

$$\frac{\partial SEI}{\partial M} = \frac{B}{T}, \quad \frac{\partial SEI}{\partial B} = \frac{M}{T}, \quad \frac{\partial SEI}{\partial T} = -\frac{BM}{T^2} \quad (3)$$

From the above equations, it can be seen that SEI is a linear function of memory (M) and brute-force scaling (B), but a quadratic function of execution time (T). Recent studies confirm that increasing memory hardness has a stronger impact on resisting hardware-accelerated attacks compared to computation alone [15]. Recent studies confirm that increasing memory hardness has a stronger impact on resisting hardware-accelerated attacks compared to computation alone [11]. This will ensure that any improvement in security parameters is meaningful and not done at the cost of computational overhead.

Second partial derivatives are useful in understanding system behavior:

$$\frac{\partial^2 SEI}{\partial M^2} = 0, \quad \frac{\partial^2 SEI}{\partial T^2} = \frac{2BM}{T^3}, \quad \frac{\partial^2 SEI}{\partial M \partial B} = \frac{1}{T} \quad (4)$$

The findings further validate linear performance in relation to memory and higher penalties for faster completion of the tasks.

3.4 Dimensional Analysis

Assigning units to each parameter:

$$[B] = \text{dimensionless}, \quad [M] = \text{MB}, \quad [T] = \text{s} \Rightarrow [SEI] = \frac{\text{MB}}{\text{s}} \quad (5)$$

Though SEI is dimensional, its main purpose is to be an index value for comparison. It is more important that one SEI be compared to another rather than the absolute value itself.

3.5 Normalised Variant

In order to provide a comparative measure that will be the same across all hardware configurations, the normalized SEI value is calculated as:

$$SEI_d = \frac{SEI}{SEI_{\text{ref}}} = \frac{B \cdot M \cdot T_{\text{ref}}}{T \cdot B_{\text{ref}} \cdot M_{\text{ref}}} \quad (6)$$

where the reference configuration refers to SHA-256. This normalization makes comparisons easier by considering how much improvement has been achieved over a benchmark.

3.6 System Architecture

The process is done through a pipeline that includes configuration, benchmarking, aggregation of metrics, computation of SEI, and visualization. The configurable hash functions are evaluated in a controlled environment, and runtime metrics are captured for execution time, memory usage, and CPU consumption. SEI is then calculated using the runtime metrics along with brute-force scaling factors, and the data is finally analyzed via visualization.

Security Performance Tradeoff Modeling of Modern Password Hashing Algorithm

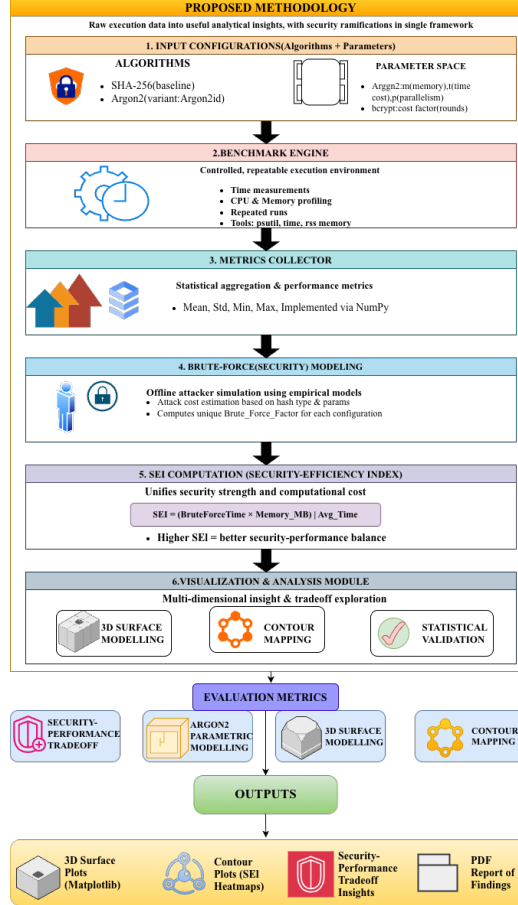


Fig. 1. Architecture of the system for the evaluation process ranging from configuration and benchmarking up until SEI calculations and visualization.

4 Experimental Setup

All experiments were carried out on a single-node system under controlled load conditions. The benchmarking framework was written in Python, using the `hashlib` library for SHA-256, the `bcrypt` library for bcrypt, the `argon2-cffi` library for Argon2, and the `passlib` library for PBKDF2. The system resource utilisation of the password hashing algorithms' memory consumption was measured by the RSS. The CPU utilisation of the algorithms was measured immediately after each run.

The study examines four different classes of password hashing algorithms. SHA-256 is used to show the weakness of non-adaptive hash functions for secure password storage. For bcrypt, the cost factors range from 6 to 16. This is due to the exponential scaling of bcrypt. Argon2 was examined over a three-dimensional grid of parameters. The memory cost parameter M takes values from the set $\{4, 8, 16, 32, 64\}$ MB. The time cost parameter t takes values from the set $\{1, 2, 3, 4\}$. The parallelism parameter p takes values from the set $\{1, 2\}$. For PBKDF2, iteration counts range from 50 000 to 800 000. This is due to the linear scaling of PBKDF2. These parameter values are based on contemporary security best practices.

Such parameter configurations are consistent with modern benchmarking practices [8]. These parameter choices are also aligned with evolving NIST recommendations for password-based key derivation [9].

For each configuration, the process was repeated $n = 30$ times to ensure reliability and stability in the measurements. In each instance, a fixed password was hashed to measure execution time. CPU usage was recorded right after executing the process, while memory consumption was derived by computing the RSS differential before and after executing the hashing process. The mean execution time was computed as

$$\mu_T = \frac{1}{n} \sum_{i=1}^n T_i \quad (7)$$

The standard deviation was computed as

$$\sigma_T = \sqrt{\frac{1}{n} \sum_{i=1}^n (T_i - \mu_T)^2} \quad (8)$$

to ensure that the variability in execution times was also estimated. Throughput was defined as

$$\text{Throughput} = \frac{1}{\mu_T} \quad (9)$$

Memory consumption was estimated based on RSS differential values, while CPU usage was recorded as the mean CPU percentages during the hashing process.

5 Results and Observations

Empirical results on performance-security tradeoffs among various password-hashing functions are discussed in this section, with special focus on the role of parameterization within Argon2.

5.1 Parametric Behaviour of Argon2

While for bcrypt and PBKDF2, the ability to scale in terms of cost is one-dimensional, Argon2 allows the separate adjustment of M , t , and p parameters

independently of each other. This flexibility significantly improves resistance against modern attack strategies [4]. This flexibility also improves resistance against hardware-accelerated attacks [11]. Thus, a greater degree of freedom is provided with regard to exploring the parameter space.

3D Surface Analysis Trends in relation to memory cost (M) and time cost (t) can be observed from the 3D surface plot of the Security Efficiency Index (SEI).

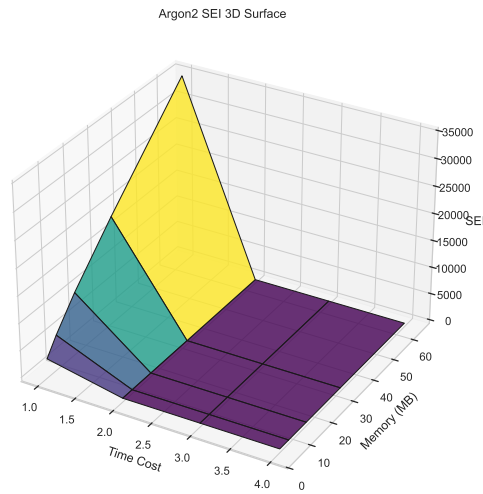


Fig. 2. Argon2 SEI 3D Surface Plot with Varying Parameters of Memory and Time Cost.

From the above Figure 2, it can be seen that SEI increases substantially as memory is increased. This means that memory hardness is the parameter which contributes most to the increase in SEI values. It should be noted that an increase in time also helps increase SEI values, although its influence is not very significant. An even greater improvement can be seen when the two variables, memory, and time, are increased concurrently.

From the surfaces above, it can be shown that the influence of memory on SEI values is mostly linear, so $\frac{\partial^2 SEI}{\partial M^2} = 0$.

Contour Analysis The contours for SEI with respect to memory and time also give some interesting observations on efficiency considerations.

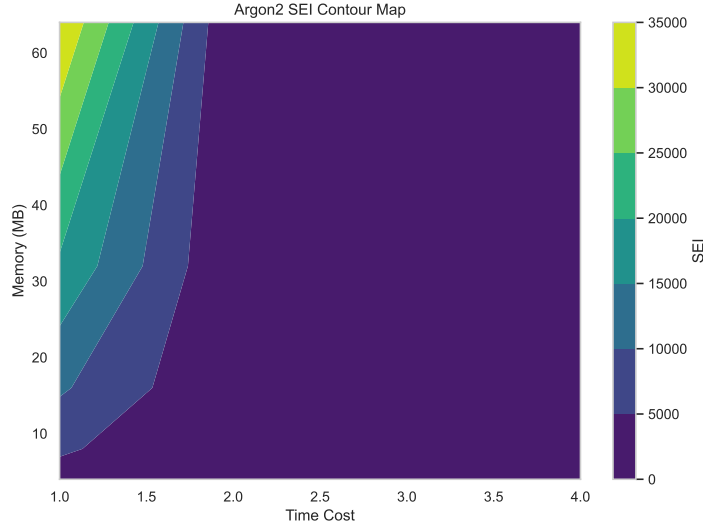


Fig. 3. SEI of Argon2 contour graph displaying iso-efficiency lines in memory-time cost space. at at at

As illustrated in Fig. 3, the contour density rises steeply with increasing memory size, especially above 16–32 MB, signifying exponential benefits in terms of security gain. However, the effect of time cost becomes increasingly insignificant beyond a certain point (i.e., $t > 3$).

This is also evident from the derivative ratio $\frac{\partial SEI}{\partial M} > \frac{\partial SEI}{\partial t}$, which confirms that memory hardness plays a more crucial role in enhancing security than iterations.

Considering practical implementations, a combination of $M \geq 16$ MB and $t = \{2, 3\}$ offers a good compromise between security and performance.

5.2 Comparative Multi-Metric Analysis

To measure the system’s performance in various criteria, radar charts comparisons were carried out among the best-performing systems based on SEI. Metrics considered include execution time, memory usage, throughput, and SEI.

All metrics were scaled using min-max normalization, which enabled comparison across different units. The obtained results reveal that Argon2 configurations requiring high memory usage, such as Argon2 ($m = 65536, t = 1, p = 1$) and ($m = 32768, t = 1, p = 1$), perform better across all criteria.

The reason is that they make full use of memory usage and produce the highest SEIs while at the same time taking less execution time. While the throughput might not be as good as fast hashing algorithms such as SHA-256, it is satisfactory enough for authentication purposes.

The algorithm with highest throughput is SHA-256, however, it uses no memory complexity and consequently has no memory hardness and low SEI, making it vulnerable to current attacks. bcrypt shows computation amplification with little memory hardness, whereas PBKDF2 relies on CPU effort.

Based on multi-objective optimization, Argon2 seems to perform much better on the Pareto-optimal frontier. Comparative studies further support the superior multi-metric performance of Argon2 [8]. Additional reviews confirm its dominance across security and efficiency dimensions [6].

6 Conclusion

Estimation of password hashing algorithm strength implies the existence of an all-encompassing characteristic that would reflect both the performance characteristics and the security provided by the algorithm. One such multi-parameter indicator is the concept of Security Efficiency Index (SEI) that combines the resistance against brute force attack, memory hardness, and latency in order to create one metric for evaluation. Benchmarking and statistical analysis have allowed to draw clear lines between different algorithms. Argon2 was found to provide the maximum SEI since it could make use of memory alongside being flexible in the amount of computational power, thus greatly increasing the difficulty for potential attackers. On the contrary, SHA-256 showed itself to be computationally efficient but lacking necessary resilience to brute force attacks; thus, it cannot be recommended for use in storing password hashes safely. While bcrypt and PBKDF2 increase security thanks to increased computational work factor, their limited memory hardness makes them less efficient than other algorithms in terms of SEI. Thus, memory utilization proved to be an even stronger contributor to security enhancement than increasing the iteration count. Argon2 can be seen as a solution that manages to balance the strength and performance aspects of password hashing successfully [4]. These findings are consistent with broader analyses of security-performance trade-offs in password hashing systems [10]. Recent security guidelines also reinforce the importance of robust password hashing practices [7].

References

1. Alwen, J., Chen, B., Pietrzak, K., Reyzin, L., Tessaro, S.: Scrypt Is maximally memory-hard. In: *Advances in Cryptology – EUROCRYPT 2017*. Lecture Notes in Computer Science, vol. 10212, pp. 33–62. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-56617-7_2
2. Bai, W., Blocki, J.: DAHash: Distribution aware tuning of password hashing costs. In: *Financial Cryptography and Data Security (FC 2021)*. Lecture Notes in Computer Science, vol. 12675, pp. 382–405. Springer, Heidelberg (2021). https://doi.org/10.1007/978-3-662-64331-0_20
3. Bai, W., Blocki, J., Ameri, M.H.: Cost-asymmetric memory hard password hashing. In: *Security and Cryptography for Networks (SCN 2022)*. Lecture

- Notes in Computer Science, vol. 13409, pp. 21–44. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-14791-3_2
4. Biryukov, A., Dinu, D., Khovratovich, D., Josefsson, S.: Argon2 memory-hard function for password hashing and proof-of-work applications. RFC 9106, Internet Research Task Force (IRTF) (Sep 2021), <https://www.rfc-editor.org/rfc/rfc9106>
 5. Blocki, J., Harsha, B., Kang, S., Lee, S., Xing, L., Zhou, S.: Data-independent memory hard functions: New attacks and stronger constructions. In: Advances in Cryptology – CRYPTO 2019. Lecture Notes in Computer Science, vol. 11693, pp. 573–607. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-26951-7_20
 6. Etese, O., Adesina, A.A.: A review and comparative analysis of password hashing techniques: Evaluating Bcrypt and Argon2. SSRN Preprint (Jul 2025). <https://doi.org/10.2139/ssrn.5363433>
 7. Fenton, J.L., Newton, E.M., Choong, Y.Y., Theofanos, M.F.: NIST special publication 800-63B-4: Digital identity guidelines — authentication and authenticator management. Special Publication 800-63B-4, National Institute of Standards and Technology (NIST) (2024). <https://doi.org/10.6028/NIST.SP.800-63B-4>
 8. Hartono, J.F., Suharjo, I.C., Sidharta, S.: Comparison of Argon2id and Bcrypt with default parameters in performance and security aspect for secure authentication. *Procedia Computer Science* **269**, 411–420 (2025). <https://doi.org/10.1016/j.procs.2025.08.293>
 9. National Institute of Standards and Technology: Decision to revise NIST SP 800-132: Recommendation for password-based key derivation, Part 1: Storage applications. NIST announcement, NIST (2023), <https://csrc.nist.gov/news/2023/proposal-to-revise-nist-sp-800-132-pbkdf>
 10. Saran, A.N.: On time-memory trade-offs for password hashing schemes. *Frontiers in Computer Science* **6**, 1368362 (2024). <https://doi.org/10.3389/fcomp.2024.1368362>
 11. Seo, H., Kim, H., Eum, S., Kwon, H.: Optimized implementation of Argon2 utilizing the graphics processing unit. *Applied Sciences* **13**(16), 9295 (2023). <https://doi.org/10.3390/app13169295>
 12. Shi, R., Zhou, Y., Li, Y., Han, W.: Understanding offline password-cracking methods: A large-scale empirical study. *Security and Communication Networks* **2021**, 5563884 (2021). <https://doi.org/10.1155/2021/5563884>
 13. Shi, R., Zhou, Y., Li, Y., Han, W.: Understanding offline password-cracking methods: A large-scale empirical study. *Security and Communication Networks* **2021**, 5563884 (2021). <https://doi.org/10.1155/2021/5563884>
 14. Tippe, P., Berner, M.P.: Evaluating Argon2 adoption and effectiveness in real-world software. arXiv preprint arXiv:2504.17121 (2025), <https://arxiv.org/abs/2504.17121>
 15. Upadhyay, D., Gaikwad, N., Zaman, M., Sampalli, S.: Investigating the avalanche effect of various cryptographically secure hash functions and hash-based applications. *IEEE Access* **10**, 112472–112486 (2022). <https://doi.org/10.1109/ACCESS.2022.3215778>
 16. Yu, F.: On deep learning in password guessing, a survey. arXiv preprint arXiv:2208.10413 (2022), <https://arxiv.org/abs/2208.10413>